

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Felipe Carrasco P

11 de septiembre de 2026

19:43

1. Introducción

En este informe analizaremos experimentalmente algoritmos para dos problemas clásicos de la computación: ordenamiento de arreglos unidimensionales y multiplicación de matrices cuadradas.

Para ambos problemas con algoritmos ampliamente conocidos y usados, el objetivo de este reporte es medir su comportamiento real y relacionarlo con su comportamiento teórico, mediante el análisis de tiempo de ejecución y uso de memoria con entradas controladas sobre 6 algoritmos distintos (4 para ordenamiento de arreglos y 2 para multiplicación de matrices). Con la final de encontrar factores que la notación asintótica no captura.

2. Entorno experimental

Todas las mediciones se realizaron en un computador portátil con las especificaciones de hardware y software que se muestran en la Tabla 1, sin otras cargas significativas, para que los tiempos reflejen el comportamiento de los algoritmos de la forma más precisa posible.

Procesador	AMD Ryzen 7 5800H with Radeon Graphics
Núcleos / Hilos	8 / 16
Frecuencia CPU min-max	3.2 GHz - 4.4 GHz
Caché L1/L2/L3	64 KiB + 512 KiB por núcleo / 16 MiB compartida
Memoria RAM	16 GB
Sistema operativo	Ubuntu 24.04.4 LTS
Compilador	g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Flags de compilación	-std=c++17 -O2 -Wall -Wextra
Versión de Python	3.12.3
Versiones de librerías	numpy 2.4.2, pandas 3.0.0, matplotlib 3.10.8

Cuadro 1: Especificaciones del hardware y software utilizado para los experimentos.

Para los datasets, se usaron las familias de entradas descritas en el enunciado de la tarea. Para ordenamiento de arreglos unidimensionales, arreglos de tamaño $n \in \{10^1, 10^3, 10^5, 10^7\}$ en 3 disposiciones (ascendente, descendente y aleatoria) y dos dominios de valores $D_1 = \{0, \dots, 9\}$ y $D_7 = \{0, \dots, 10^7\}$. Para multiplicación de matrices cuadradas, se usaron matrices de tamaño $n \times n$ con $n \in \{2^4, 2^6, 2^8, 2^{10}\}$ de 3 tipos (densa, diagonal y dispersa) y dos dominios de valores $D_0 = \{0, 1\}$ y $D_{10} = \{0, \dots, 9\}$. Cada configuración tiene 3 muestras aleatorias independientes, generadas con numpy mediante una semilla fija, de modo que todos los datos son reproducibles. Por su tamaño (~1,6 GB) los datasets no se incluyen en la entrega, solo el generador y la semilla.

Cada medición corre en un proceso separado, para poder aislar el tiempo y la memoria de cada algoritmo de los demás. El tiempo se mide con la librería chrono de C++ alrededor de la llamada al algoritmo sin contar la escritura ni lectura de los archivos. La memoria se mide como el pico de RSS del proceso (getrusage, campo ru_maxrss, en kB) menos una línea base tomada con la copia de trabajo ya en memoria; así el valor reportado corresponde a la memoria auxiliar que usa el algoritmo, sin contar la entrada. Cada medición tiene un límite de tiempo; al superarlo se marca como NA (por ejemplo, QuickSort en su caso cuadrático para $n = 10^7$). El resultado de cada configuración es el promedio de sus tres muestras.

El código fuente, los scripts de medición y los generadores de gráficos están disponibles en <https://github.com/fcarrascop/Tarea-1-Algoritmos-y-Complejidad>.

3. Resultados y Análisis

Los valores entregados son el promedio de las 3 muestras de cada configuración. Como cada variación entre las muestras fue baja (inferior al 4% en todos los casos), no se incluyen barras de error. Cada figura se genera automáticamente con el script `plot_generator.py` correspondiente; una copia queda en `code/sorting/data/plots/` y `code/matrix_multiplication/data/plots/`.

3.1. Ordenamiento de arreglos

Las implementaciones de Merge Sort y QuickSort se tomaron de GeeksforGeeks [3, 4], la de Patience Sort de Rosetta Code [6] y `std::sort` es la de la biblioteca estándar [2]. QuickSort se modificó para usar pivote aleatorio [1, cap. 7].

3.1.1. Tiempo de ejecución

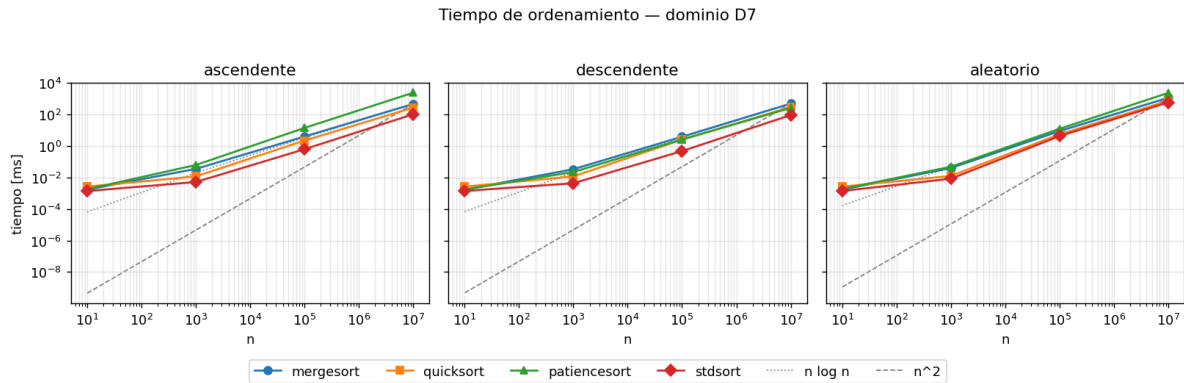


Figura 1: Tiempo de ejecución promedio de los cuatro algoritmos de ordenamiento en el dominio $D_7 = \{0, \dots, 10^7\}$, por tipo de entrada. Escala log-log; las líneas grises son las referencias $n \log n$ y n^2 .

n	$D_1 = \{0, \dots, 9\}$				$D_7 = \{0, \dots, 10^7\}$			
	Merge	Quick	Patience	std	Merge	Quick	Patience	std
10^3	0.039	0.041	0.037	0.006	0.039	0.012	0.047	0.008
10^5	5.81	235.2	4.97	1.52	8.75	5.44	12.09	4.18
10^7	636	—	487	172	1125	722	2304	576

Cuadro 2: Tiempo de ordenamiento [ms], entrada aleatoria. —: la corrida superó el límite de tiempo de 300 s.

Con respecto a D_7 (fig. 1, tabla 2), todos siguen la pendiente de $n \log n$. Sin embargo, podemos ver que `std::sort` es el más rápido y el Patience Sort es el más lento para n grande (2304 ms vs 576 ms de `std::sort` a $n = 10^7$). Podemos ver también que los factores de crecimiento medidos de 10^5 a 10^7 son $\times 129$ (Merge), $\times 133$ (Quick), $\times 138$ (std) y $\times 191$ (Patience), cercanos al $\times 140$ que predice $n \log n$. La desviación de Patience Sort se explica por su mayor constante y su peor localidad de memoria (reparto en pilas y mezcla con un heap).

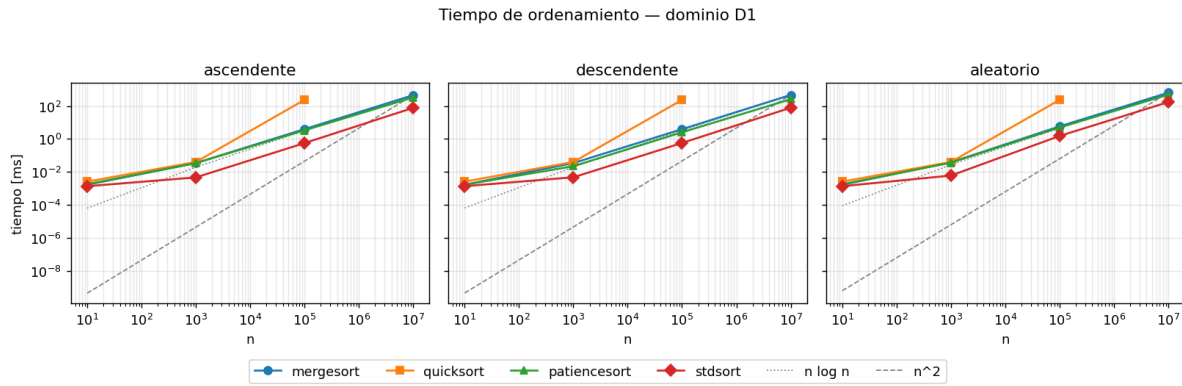


Figura 2: Tiempo de ejecución promedio en el dominio $D_1 = \{0, \dots, 9\}$. QuickSort se separa del resto y a $n = 10^7$ no completa dentro del límite de tiempo (su curva termina en $n = 10^5$).

El comportamiento cambia por completo en D_1 (fig. 2): QuickSort se curva hacia n^2 y a $n = 10^7$ no termina. Ya a $n = 10^5$ tarda 235 ms, unas 40 veces más que Merge Sort. La causa es la enorme repetición de valores: con solo 10 valores distintos, tras unos pocos niveles de recursión un subarreglo queda formado por un único valor repetido, y la partición de Lomuto, con su comparación estricta ($<$), extrae un solo elemento por llamada, degradándose a $O(m^2)$. Como el pivote es aleatorio, esto no es el peor caso clásico de entrada ordenada (de hecho las entradas ascendente y descendente en D_7 terminaron sin problema), sino un efecto propio de los datos con muchos duplicados que ningún pivote corrige.

3.1.2. Uso de memoria

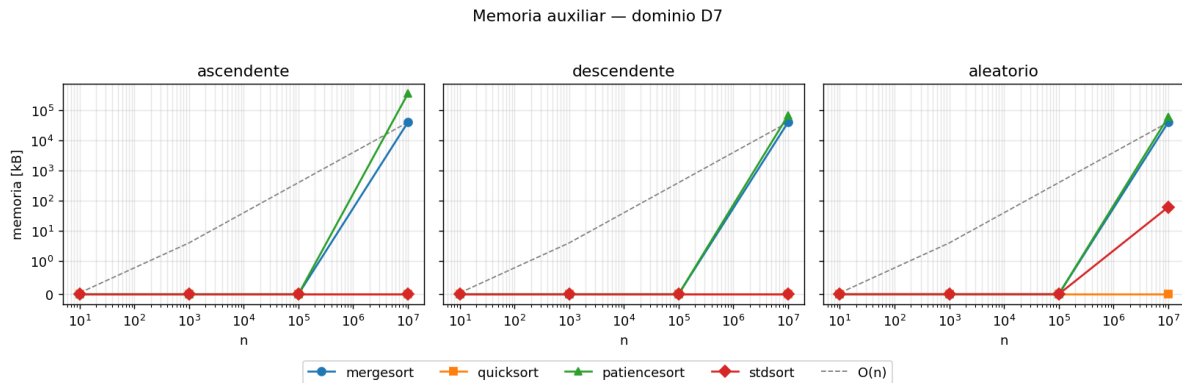


Figura 3: Memoria auxiliar (pico de RSS menos línea base) de los algoritmos de ordenamiento en D_7 . Escala *symlog*: los algoritmos in-place se dibujan sobre el eje.

Algoritmo	Memoria aux. [kB]	Complejidad esperada
QuickSort	0	$O(\log n)$ (pila)
<code>std::sort</code>	60	$O(\log n)$
MergeSort	38 912	$O(n)$ ($\approx n \cdot 4$ B)
PatienceSort	58 240	$O(n)$ ($\approx 1,5 n$)

Cuadro 3: Memoria auxiliar a $n = 10^7$, entrada aleatoria, dominio D_7 .

La [fig. 3](#) y la [tabla 3](#) muestran dos grupos claros. QuickSort y `std::sort` operan in-place y su memoria auxiliar es prácticamente nula. Merge Sort y Patience Sort crecen linealmente con n : Merge necesita el arreglo auxiliar de la mezcla ($\approx n \cdot 4$ bytes) y Patience almacena las pilas ($\approx 1,5 n$). Los valores medidos coinciden con la complejidad espacial teórica de cada algoritmo.

3.2. Multiplicación de matrices

Tanto el algoritmo naïve como Strassen [7] se tomaron de GeeksforGeeks [5].

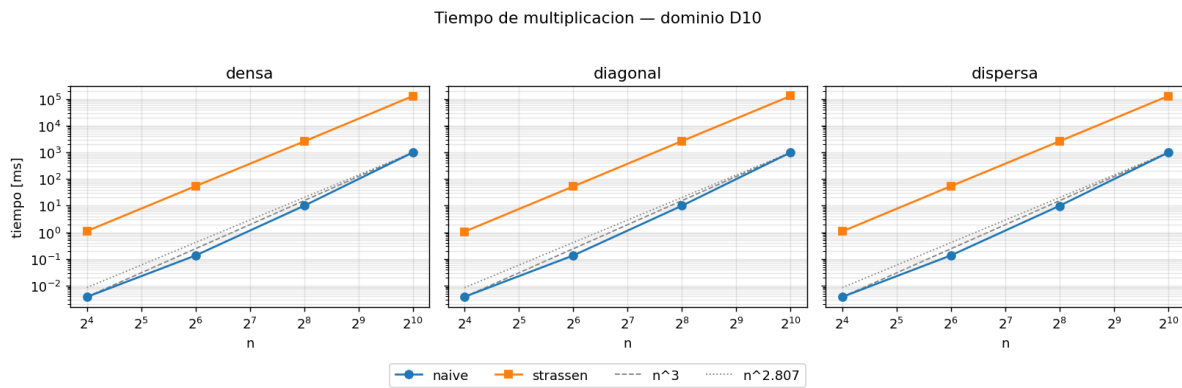


Figura 4: Tiempo de multiplicación de matrices en el dominio $D_{10} = \{0, \dots, 9\}$, por tipo de matriz. Escala log-log; referencias n^3 y $n^{2.807}$.

n	Naïve [ms]	Strassen [ms]	Naïve mem [kB]	Strassen mem [kB]	Ratio
2^4	0.004	1.12	0	0	296×
2^6	0.14	53.97	0	0	399×
2^8	9.97	2 581	0	0	259×
2^{10}	995	129 718	3 849	49 841	130×

Cuadro 4: Multiplicación de matrices, tipo densa, dominio D_{10} . El tipo (densa / diagonal / dispersa) no altera los resultados.

En [fig. 4](#) y [tabla 4](#) el algoritmo naïve sigue la referencia n^3 , mientras que Strassen queda unos dos órdenes de magnitud por encima pero con pendiente algo menor. A $n = 1024$, naïve tarda 1 s y Strassen 130

s. El factor de crecimiento al duplicar n es 7,0/6,9/7,1 para Strassen (muy cercano al 7 que corresponde a $n^{\log_2 7} = n^{2,807}$) y 6,0/8,6/10,0 para naïve (en torno al 8 de n^3). Es decir, Strassen *sí* tiene la mejor complejidad asintótica y se observa en la pendiente, pero su constante lo hace 130 veces más lento: el *ratio* baja de $296\times$ a $130\times$ al crecer n , la curva converge, pero el punto de cruce está fuera de todo rango práctico. La constante proviene de copiar los `std::vector` anidados que representan las matrices y de pasar las submatrices por valor en cada uno de los siete productos recursivos.

Dos observaciones adicionales. Primero, naïve crece *más* rápido que n^3 entre 2^8 y 2^{10} ($\times 10$ por duplicación): a $n = 1024$ cada matriz ocupa 4 MB y las tres no caben cómodamente en la caché L3 (16 MB), lo que aumenta los fallos de caché (un efecto que la notación asintótica no captura). Segundo, el tipo de matriz (densa, diagonal o dispersa) no cambia los tiempos: ambos algoritmos operan sobre almacenamiento denso y no aprovechan la estructura.

4. Conclusiones

En base a los datos obtenidos de las mediciones, se puede concluir que la complejidad asintótica predice bien la forma de las curvas, pero no nos cuenta toda la historia. En el ordenamiento los cuatro algoritmos crecieron como $n \log n$ en D_7 , pero aun así `std::sort` fue 4 veces más rápido que Patience Sort a $n = 10^7$ (576 vs 2304 ms), por la diferencia de constante y de localidad de memoria: por más que las cotas sean similares, la implementación y el uso de recursos son distintos. Con las matrices ocurre lo mismo, pero de forma más extrema. Strassen creció con factor 7,0 al duplicar n , exactamente lo que predice $O(n^{\log_2 7})$, y sin embargo a $n = 1024$ tardó 130 s frente a 1 s de naïve y usó 50 MB de memoria auxiliar vs 4 MB, porque el costo de copiar submatrices en cada llamada pesa más que las operaciones que ahorra. La diferencia se va acortando al crecer n ($296\times \rightarrow 130\times$), pero el cruce queda fuera de nuestra medición: para matrices de tamaño pequeño y mediano, Strassen no es competitivo frente al algoritmo ingenuo.

Referencias

- [1] Thomas H. Cormen et al. *Introduction to Algorithms*. 3.^a ed. MIT Press, 2009.
- [2] cppreference.com. `std::sort`. <https://en.cppreference.com/w/cpp/algorithm/sort>. Consultado en septiembre de 2026.
- [3] GeeksforGeeks. *Merge Sort - Data Structure and Algorithms Tutorials*. <https://www.geeksforgeeks.org/dsa/merge-sort/>. Consultado en septiembre de 2026.
- [4] GeeksforGeeks. *Quick Sort*. <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>. Consultado en septiembre de 2026.
- [5] GeeksforGeeks. *Strassen's Matrix Multiplication*. <https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/>. Consultado en septiembre de 2026.

- [6] Rosetta Code. *Sorting algorithms/Patience sort*. https://rosettacode.org/wiki/Sorting_algorithms/Patience_sort. Implementación en C++. Consultado en septiembre de 2026.
- [7] Volker Strassen. «Gaussian elimination is not optimal». En: *Numerische Mathematik* 13.4 (1969), págs. 354-356.